Документация библиотеки Big Integer Arithmetic

Полное руководство по использованию

Разработчик: Карпихин Вадим

29 мая 2026 г.

Содержание

1 Введение **2**
- 1.1 Основные возможности . . . 2
- 1.2 Архитектура . . . 2

2 Внутреннее представление **6**
- 2.1 Формат хранения `Int_Number` . . . 6
- 2.2 Глобальная функция `Length_Int<T>` . . . 6

3 Класс `Int_Number` **7**
- 3.1 Конструкторы . . . 7
- 3.2 Публичные методы . . . 7
- 3.3 Операторы . . . 7
  - 3.3.1 Сравнение (через `<=>`) . . . 7
  - 3.3.2 Арифметика . . . 7
  - 3.3.3 Дружественные функции . . . 8
- 3.4 Создание рационального числа . . . 8

4 Класс `Quotient_Number` **9**
- 4.1 Структура . . . 9
- 4.2 Конструкторы . . . 9
- 4.3 Методы . . . 9
- 4.4 Операторы . . . 9

5 Многопоточность и производительность **11**
- 5.1 Thread_Pool . . . 11
- 5.2 NTT-умножение . . . 11

6 Обработка ошибок **12**

7 Примеры использования **13**
- 7.1 Базовые операции . . . 13
- 7.2 Факториал 100 . . . 13
- 7.3 Параллельное сложение . . . 13

8 Сборка и интеграция **14**
- 8.1 Требования . . . 14

9 Отладка **15**
- 9.1 Методы диагностики . . . 15
- 9.2 Формат вывода `All_Print` . . . 16

10 Ограничения и рекомендации **17**

1 Введение

Библиотека `Big Integer Arithmetic` реализует систему для работы с целыми и рациональными числами на языке C++ с поддержкой многопоточных вычислений.

1.1 Основные возможности

- **Целые числа произвольной длины** — хранение чисел любой величины в базе 10^9
- **Рациональные числа** — точные вычисления с дробями и сокращением
- **Полный набор арифметических операций** — $+$, $-$, $\times$, $\div$, $\%$, сдвиги
- **Сравнение** — все операторы через трёхстороннее сравнение (`<=>`)
- **Многопоточность** — параллельное сложение/вычитание через `Thread_Pool`
- **NTT-умножение** — быстрое умножение больших чисел через теорему об остатках
- **Цветной вывод** — удобная отладка через ANSI-коды

1.2 Архитектура

Структура библиотеки

- `Int_Number` — целые числа абсолютной точности
- `Quotient_Number` — рациональные числа (на основе `Int_Number`)
- `Thread_Pool` — пул потоков для параллельных операций
- `Service_Names.h` — константы, шаблон `Length_Int<T>`, модули для NTT
- `Error.h` — иерархия классов для обработки ошибок (без исключений)



Рис. 1: Блок-схема Int_Number

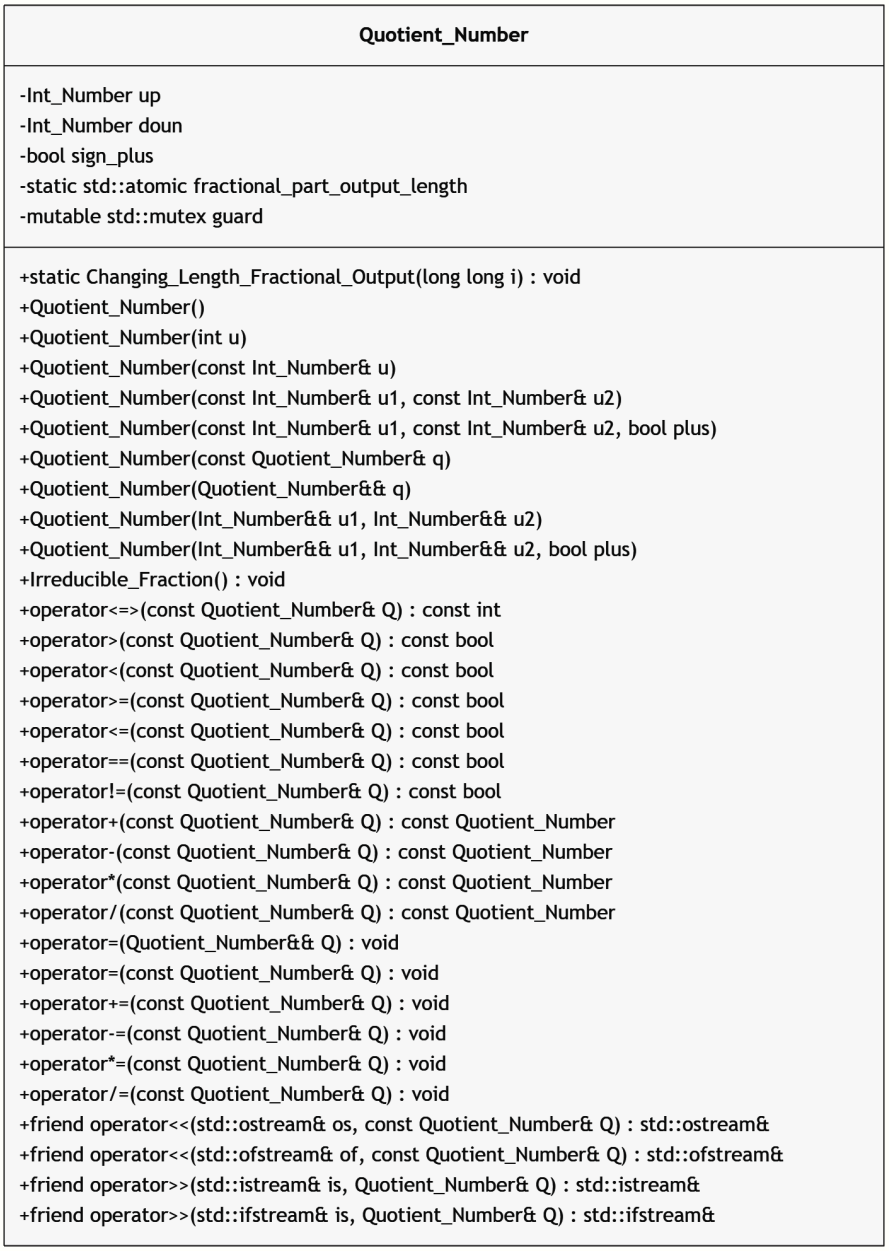


Рис. 2: Блок-схема Quotient_Numberr

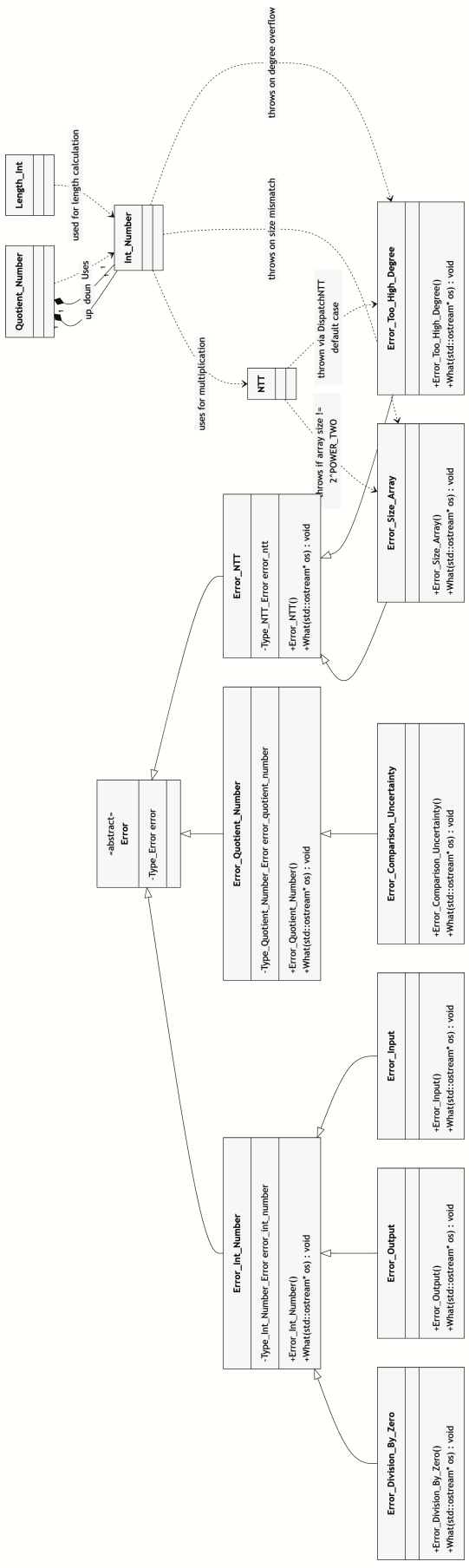


Рис. 3: Полная блок-схема

2 Внутреннее представление

2.1 Формат хранения `Int_Number`

Число хранится в виде массива 32-битных блоков, где каждый блок содержит до 9 десятичных цифр (основание 10^9).

Таблица 1: Поля класса `Int_Number`

Поле	Тип	Описание
`num`	`int*`	Динамический массив блоков числа
`ptr_head`	`int`	Индекс старшего (наиболее значимого) блока
`plt_tail`	`int`	Индекс младшего блока (*обратите внимание: имя поля — `plt_tail`*)
`len_dedicated`	`int`	Выделенный размер массива
`len_num`	`long`	Фактическая длина числа в десятичных цифрах
`sign_plus`	`bool`	`true` — положительное, `false` — отрицательное
`guard`	`std::mutex`	Мьютекс для потокобезопасности

```
Пример: число 12 345 678 901 234 567 890

Блок 0 (plt_tail): 234567890  ← младшие 9 цифр
Блок 1:            345678901  ← следующие 9 цифр
Блок 2 (ptr_head): 12         ← старшие 2 цифры
```

2.2 Глобальная функция `Length_Int<T>`

Листинг 1: Шаблон для подсчёта цифр

```
template<typename T>
int Length_Int(T num) {
    if (num == 0) return 1;
    T n = num < 0 ? -num : num;
    if (n < 10) return 1;
    if (n < 100) return 2;
    if (n < 1000) return 3;
    if (n < 10000) return 4;
    return (long)log10(n) + 1;  //для больших чисел
}
```

Возвращает

Количество десятичных цифр в числе `num` без учёта знака.

3 Класс Int_Number

3.1 Конструкторы

Таблица 2: Конструкторы Int_Number

Сигнатура	Описание
`Int_Number()`	Число 0
`Int_Number(int len_dedicated1, int plt_tail1)`	Конструктор с параметрами памяти
`Int_Number(int n)`	Из int
`Int_Number(long long int n)`	Из long long
`Int_Number(const std::string & line_num)`	Из строки
`Int_Number(const Int_Number & Nu)`	Конструктор копирования
`Int_Number(Int_Number && Nu)`	Конструктор перемещения

3.2 Публичные методы

Листинг 2: Основные методы

```
// Отладка
void All_Print(int count = 10) const;        // Детальный вывод структуры
void Technical_Print() const;                // Техническая информация

// Управление памятью
void Relocation(int count = 3);              // Перераспределение памяти

// Свойства
bool Is_plus() const;                        // Знак числа
int Length_Number() const;                   // Количество цифр

// Преобразование
std::string To_String() const;               // В строку с форматированием
std::string To_Clear_String() const;         // В строку без форматирования
```

3.3 Операторы

3.3.1 Сравнение (через <=>)

```
int operator <=> (const Int_Number & Nu) const;  // трёхстороннее сравнение
// Возвращает: 1 если *this > Nu, -1 если <, 0 если ==

// На основе <=> реализованы:
bool operator >, <, >=, <=, ==, != (const Int_Number & Nu) const;
```

3.3.2 Арифметика

Листинг 3: Арифметические операторы

```
// Унарные
Int_Number & operator - ();  // смена знака

// С присваиванием
void operator +=, -=, *=, /=, %= (const Int_Number & Nu);
void operator *= (int q);

// Бинарные возвращают( новый объект)
Int_Number operator +, -, *, /, % (const Int_Number & Nu) const;
Int_Number operator * (int q) const;

// Сдвиги на( блоки по 9 цифр)
Int_Number & operator << (int i);  // умножение на 10^(9*i)
Int_Number & operator >> (int i);  // деление на 10^(9*i)
```

Важно!

Операторы « и » сдвигают число на **блоки**, а не на биты. Один сдвиг = умножение/деление на 10^9.

3.3.3 Дружественные функции

```
// Модуль числа изменяет аргумент
friend Int_Number & Abs(Int_Number & Nu);

// НОД алгоритм Евклида
friend Int_Number NOD(const Int_Number & Nu1, const Int_Number & Nu2);

// Ввод
friend std::ostream & operator << (std::ostream & os, const Int_Number & Nu)
    ;
// Вывод
friend std::istream & operator >> (std::istream & is, Int_Number & Nu);
```

3.4 Создание рационального числа

```
// Оператор | создаёт дробь из двух целых чисел
Quotient_Number operator | (const Int_Number & Nu1) const;

// Пример:
Int_Number a(3), b(4);
Quotient_Number frac = a | b;  // дробь 3/4
```

4 Класс Quotient_Number

4.1 Структура

Таблица 3: Поля Quotient_Number

Поле	Тип	Описание
up	Int_Number	Числитель
doun	Int_Number	Знаменатель
sign_plus	bool	Знак дроби
guard	std::mutex	Мьютекс для корректности операций

4.2 Конструкторы

```
Quotient_Number();                                          // 0/1
Quotient_Number(int u);                                     // u/1
Quotient_Number(const Int_Number & u);                      // u/1
Quotient_Number(const Int_Number & u1,
                const Int_Number & u2);                     // u1/u2
Quotient_Number(const Int_Number & u1,
                const Int_Number & u2, bool plus);          // с явным знаком
// Поддержка перемещения:
Quotient_Number(Int_Number && u1, Int_Number && u2);
```

4.3 Методы

```
// Сокращение дроби до несократимого вида
void Make_Irreducible_Fraction();

// Управление длиной дробной части при выводе
static void Set_Precision(long long i);
```

4.4 Операторы

- **Сравнение**: <=>, ==, !=, <, >, <=, >=
- **Арифметика**: +, -, *, / (с автоматическим сокращением)
- **Присваивание**: =, +=, -=, *=, /=
- **Унарный минус**: operator - ()
- **Ввод/вывод**: «, »

Листинг 4: Пример работы с дробями

```
Quotient_Number a(1, 2);    // 1/2
Quotient_Number b(1, 3);    // 1/3

Quotient_Number sum = a + b;           // 5/6
Quotient_Number prod = a * b;          // 1/6
Quotient_Number quot = a / b;          // 3/2

sum.Set_Precision();  // можно вызвать явно
std::cout << sum;             // вывод: 0 [.] 833_333_333  333_333_333 ...
```

5 Многопоточность и производительность

5.1 Thread_Pool

Библиотека использует внутренний пул потоков для ускорения операций сложения и вычитания больших чисел.

Как это работает

1. Блоки распределяются между потоками
2. Результаты объединяются с учётом переносов

5.2 NTT-умножение

Для умножения очень больших чисел используется **Number Theoretic Transform** с китайской теоремой об остатках.

Таблица 4: Модули для NTT

Модуль	Значение	Макс. степень 2
Mod998244353	998 244 353	2^{23}
Mod1004535809	1 004 535 809	2^{21}
Mod469762049	469 762 049	2^{26}
Mod167772161	167 772 161	2^{25}

Листинг 5: Автоматический выбор количества модулей

```
    int count_modile = 3;
    int power_2 = 6;
    while((1 << power_2) < (len_num/9 + Nu.len_num/9 + 2))
        power_2++;

    if((1 << power_2) > 470000000)
        count_modile = 4;

    if(count_modile == 3){...}
    if(count_modile == 4){...}
```

Когда используется NTT?

Библиотека автоматически выбирает алгоритм:

- **Обычное умножение** — для чисел до 32 блоков
- **NTT + CRT** — для больших чисел (автоматически)

Пользователю не нужно выбирать алгоритм вручную.

6 Обработка ошибок

Библиотека **не использует исключения** (`throw`). Вместо этого реализована иерархия классов ошибок:

Листинг 6: Иерархия ошибок

```
Error
 Error_Int_Number
   Error_Division_By_Zero
   Error_Output
   Error_Input
 Error_Quotient_Number
   Error_Comparison_Uncertainty
 Error_NTT
    Error_Size_Array
    Error_Too_High_Degree
```

Рекомендация

Всегда проверяйте делитель на ноль перед операцией `/` или `%`.

7 Примеры использования

7.1 Базовые операции

Листинг 7: Hello Big Integer

```cpp
#include "Library/Int_Number/Int_Number.h"
#include "Library/Quotient_Number/Quotient_Number.h"
#include <iostream>

int main() {
    Quotient_Number::Set_Precision(18);

    Int_Number a(123456789);
    Int_Number b("98765432109876543210");  // из строки

    Int_Number sum = a + b;
    std::cout << "Сумма: " << sum << '\n';

    // Дробь
    Quotient_Number frac = a | b;
    frac.Irreducible_Fraction();
    std::cout << "Дробь: " << frac << '\n';

    // НОД
    Int_Number g = NOD(a, b);
    std::cout << "НОД: " << g << '\n';

    return 0;
}
```

```
Сумма: 98765432109876543210 + 123456789 = 98765567765543320999
Дробь: 0 [.] 000_000_000 000_124_999 ...
НОД: 3
```

7.2 Факториал 100

Листинг 8: Вычисление 100!

```cpp
Int_Number factorial(1);
for (int i = 2; i <= 100; ++i)
    factorial *= i;

std::cout << "100! имеет " << factorial.Length_Number() << " цифр\n";
```

```
100! имеет 158 цифр
```

7.3 Параллельное сложение

Листинг 9: Автоматическое использование потоков

```cpp
Int_Number huge1("12345678901234567890...");  // очень длинное
Int_Number huge2("98765432109876543210...");

// При сложении библиотека автоматически задействует Thread_Pool,
// если число блоков превышает порог
Int_Number result = huge1 + huge2;  // параллельно, если нужно
```

8 Сборка и интеграция

8.1 Требования

- Компилятор с поддержкой C++26 (для `std::scoped_lock`, `<=>`)
- Потоки: `-pthread` (GCC/Clang) или `/threads` (MSVC)
- Терминал с поддержкой ANSI-кодов для цветного вывода

9 Отладка

9.1 Методы диагностики

Листинг 10: Инструменты отладки

```
Int_Number num(123456789);

// Полная структура цветной( вывод)
num.All_Print();       // 10 блоков на строку
num.All_Print(2);      // 2 блока на строку

// Техническая информация
num.Technical_Print();

// Свойства
std::cout << "Знак:␣" << (num.Is_plus() ? "+" : "-") << '\n';
std::cout << "Длина:␣" << num.Length_Number() << "␣цифр\n";

// Строковое представление
std::string s1 = num.To_String();         // с подчёркиваниями: 123_456_789
std::string s2 = num.To_Clear_String();   // чистый: 123456789
```

```
+ 000_000_000  123_456_789  000_000_000

ptr_head = 1
plt_tail = 1
len_dedicated = 3
len_num = 9

+ 000_000_000  123_456_789
000_000_000

ptr_head = 1
plt_tail = 1
len_dedicated = 3
len_num = 9

ptr_head = 1
plt_tail = 1
len_dedicated = 3
len_num = 9

Знак: +
Длина: 9
```

9.2 Формат вывода `All_Print`

1. Знак: `+` или `-`
2. Блоки по 9 цифр, разделённые пробелами
3. Цвета (если терминал поддерживает):
   - Зелёный — используемые блоки
   - Жёлтый — зарезервированные, но пустые
4. Служебная информация: `ptr_head`, `plt_tail`, `len_dedicated`

10 Ограничения и рекомендации

Важные замечания

- **Память** — при очень больших числах следите за потреблением памяти
- **NTT** — требует, чтобы размер результата был степенью двойки; библиотека обрабатывает это автоматически

Советы по производительности

- Используйте `std::move` при возврате больших чисел из функций
- Для последовательных операций предпочитайте `+=`, `*=` вместо `+`, `*`
- Для вывода в файл используйте `To_Clear_String()` — быстрее, чем потоковый оператор